

Diseño de Sistemas Informáticos **con UML**

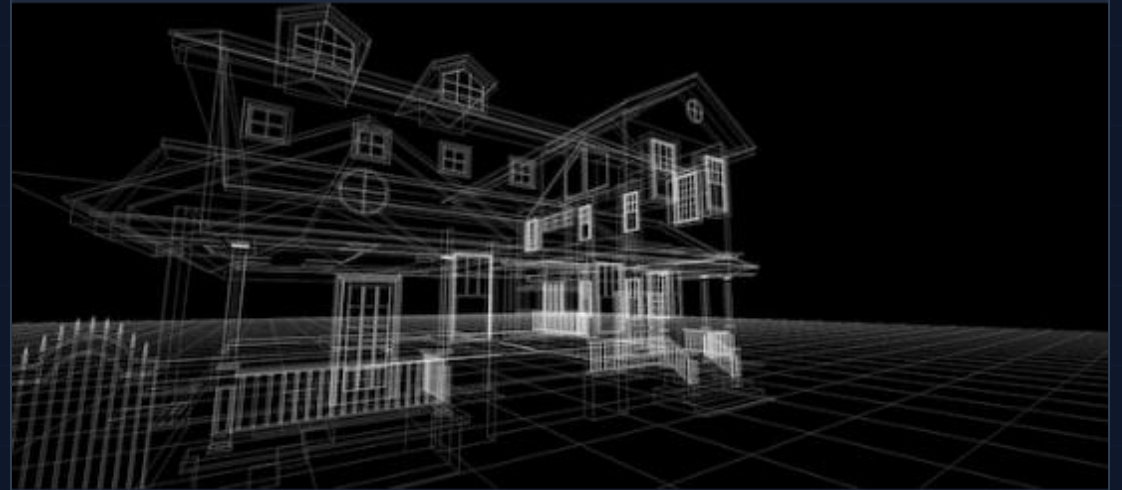
Una guía completa de modelado, análisis y arquitectura de software utilizando
el Lenguaje Unificado de Modelado.

Fundamentos del Diseño

¿Por qué diseñar antes de codificar?

El diseño de sistemas es el proceso de definir la arquitectura, componentes, módulos, interfaces y datos de un sistema para satisfacer requisitos específicos.

- > Reduce la complejidad mediante la **abstracción**.
- > Mejora la **comunicación** entre stakeholders.
- > Facilita el **mantenimiento** y la escalabilidad.
- > Permite detectar errores en etapas tempranas (menos costoso).



Introducción a UML

Unified Modeling Language

UML no es una metodología, es un **lenguaje estándar** para visualizar, especificar, construir y documentar los artefactos de un sistema de software.

- > **Estandarizado** por la OMG (Object Management Group).
- > Independiente del lenguaje de programación (Java, C#, Python...).
- > Utiliza una notación gráfica común para todos los ingenieros.



UML 2.5

Clasificación de Diagramas

Dos Grandes Categorías

UML 2.x divide los diagramas en dos grupos principales que responden a preguntas diferentes sobre el sistema.

🏗️ Estructurales (Estáticos)

Muestran la organización del sistema, la arquitectura y los datos. "Lo que el sistema ES". Ej: Clases, Componentes, Despliegue.

▶ Comportamiento (Dinámicos)

Muestran la ejecución, flujo y cambios de estado. "Lo que el sistema HACE". Ej: Casos de Uso, Secuencia, Actividades.

Estructura

- > Clases
- > Objetos
- > Componente
- > Despliegue

Comportamiento

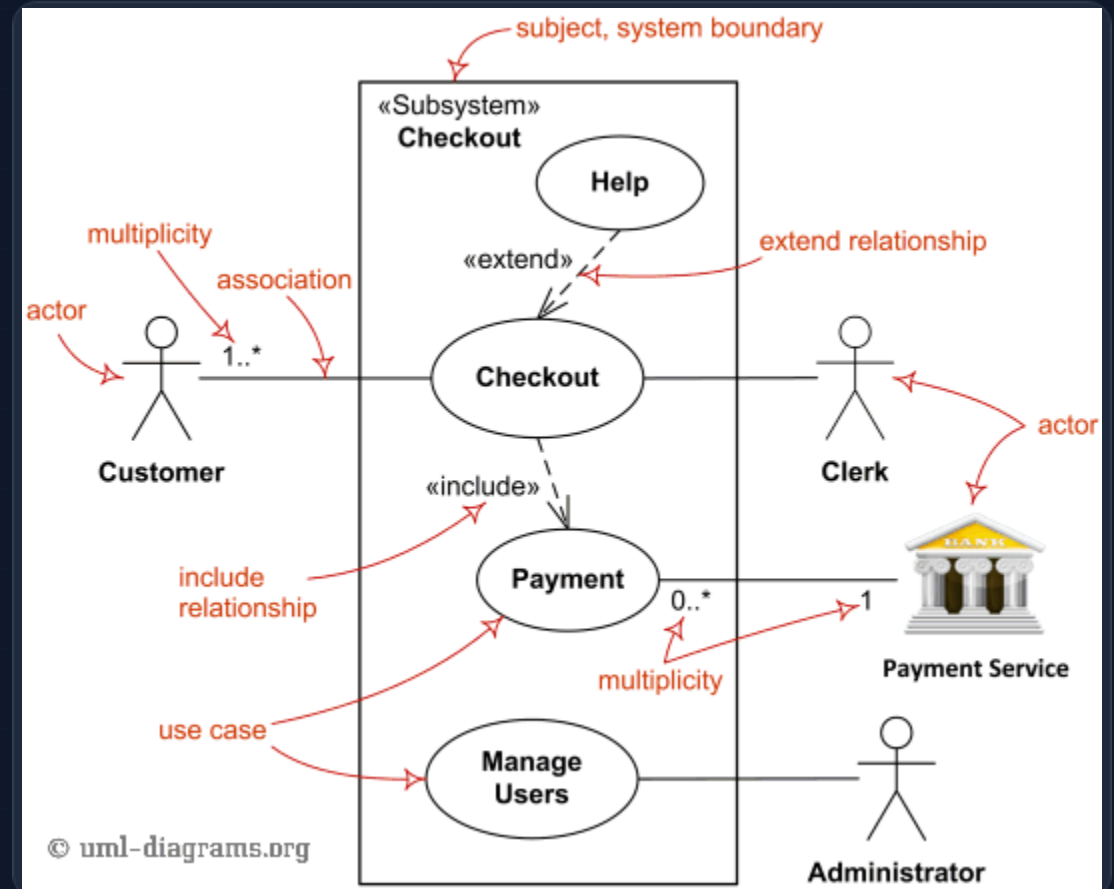
- > Casos de Uso
- > Secuencia
- > Actividades
- > Estados

Diagramas de Casos de Uso

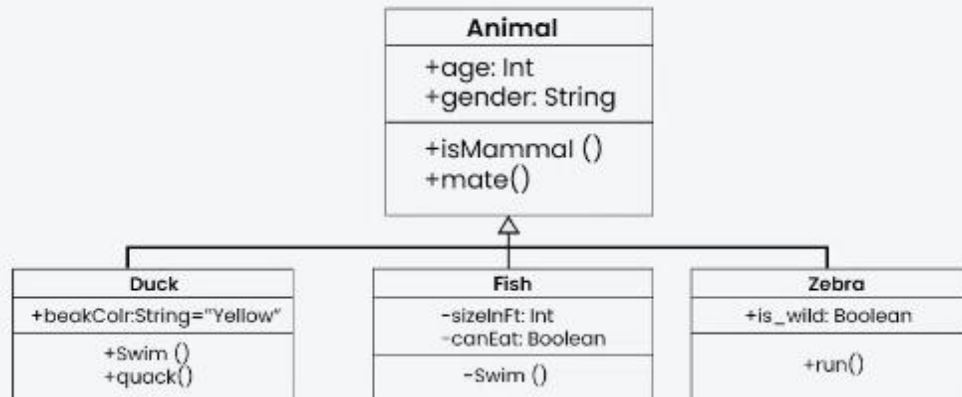
Requerimientos Funcionales

Describen qué hace el sistema desde el punto de vista del usuario, sin entrar en detalles técnicos de implementación.

- > **Actor:** Entidad externa (Usuario, otro sistema).
- > **Caso de Uso:** Funcionalidad específica (el óvalo).
- > **Relaciones:** Include (obligatorio), Extend (opcional).
- > **Objetivo:** Definir el alcance del sistema.



Diagramas de Clases



Class Diagram example



La columna vertebral del diseño

Es el diagrama más utilizado. Modela la estructura estática del sistema y sirve de base para la generación de código.

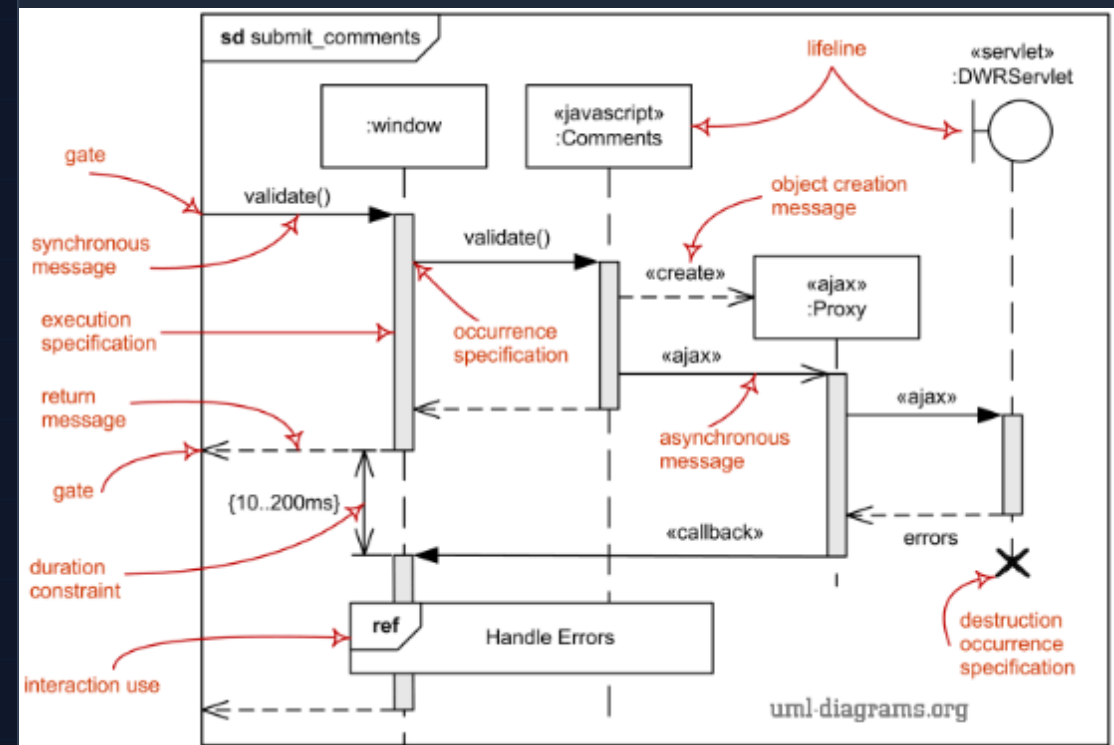
- > **Clase:** Rectángulo con Nombre, Atributos y Métodos.
- > **Visibilidad:** + Público, - Privado, # Protegido.
- > **Relaciones:**
 - Herencia (Generalización)
 - ◆— Composición (Todo-Parte fuerte)
 - ◇— Agregación (Todo-Parte débil)

Diagramas de Secuencia

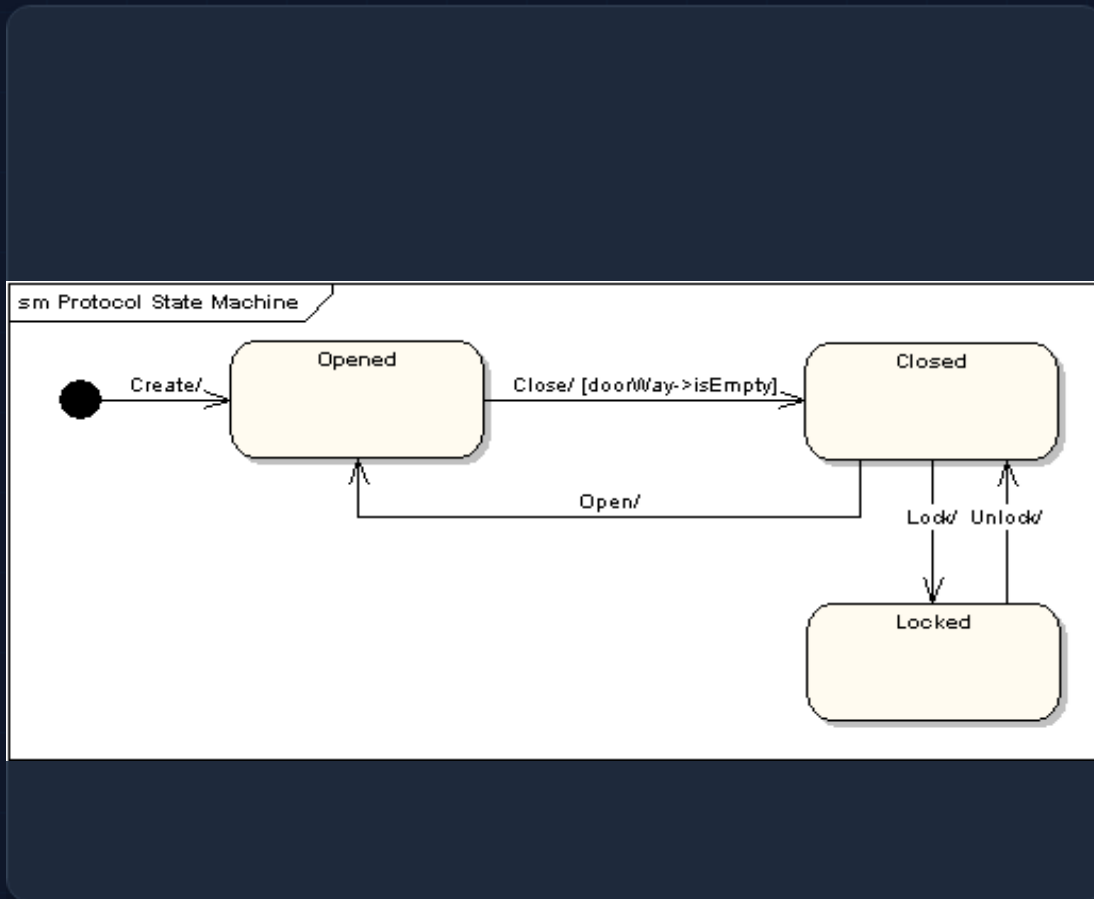
Interacción paso a paso

Muestra cómo los objetos interactúan entre sí en un orden temporal específico para completar un caso de uso.

- > **Línea de vida (Lifeline):** Representa la existencia del objeto.
- > **Mensajes:** Síncronos (línea sólida) y Asíncronos (línea abierta).
- > **Fragmentos combinados:** Bucles (loop), Alternativas (alt/opt).
- > Ideal para modelar lógica compleja de protocolos.



Diagramas de Estados



Máquinas de Estado Finitos

Modelan el comportamiento de un objeto individual a lo largo de su vida, reaccionando a eventos.

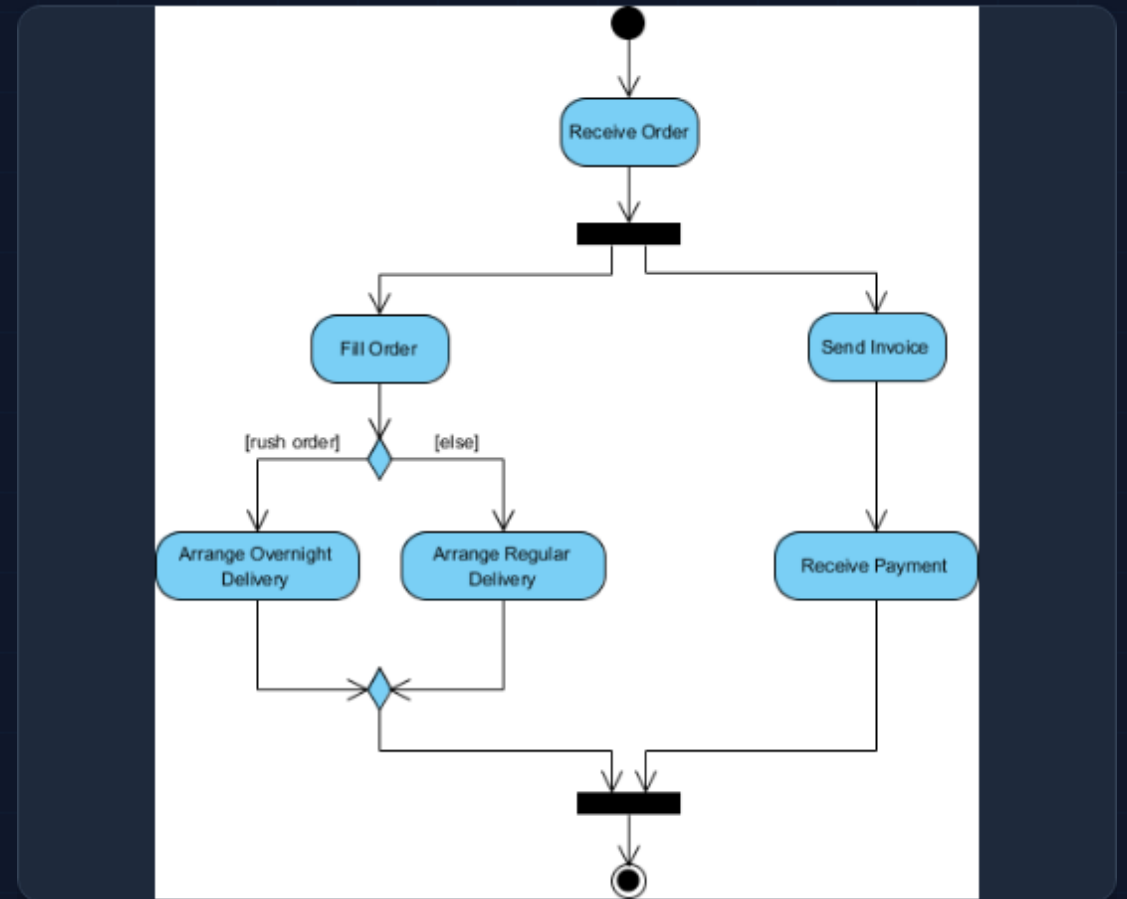
- > **Estado:** Condición o situación en la vida de un objeto (ej: Pendiente, Aprobado, Cancelado).
- > **Transición:** Cambio de un estado a otro disparado por un evento.
- > **Esencial para:** Sistemas reactivos, protocolos de comunicación, ciclos de vida de documentos o pedidos.

Diagramas de Actividades

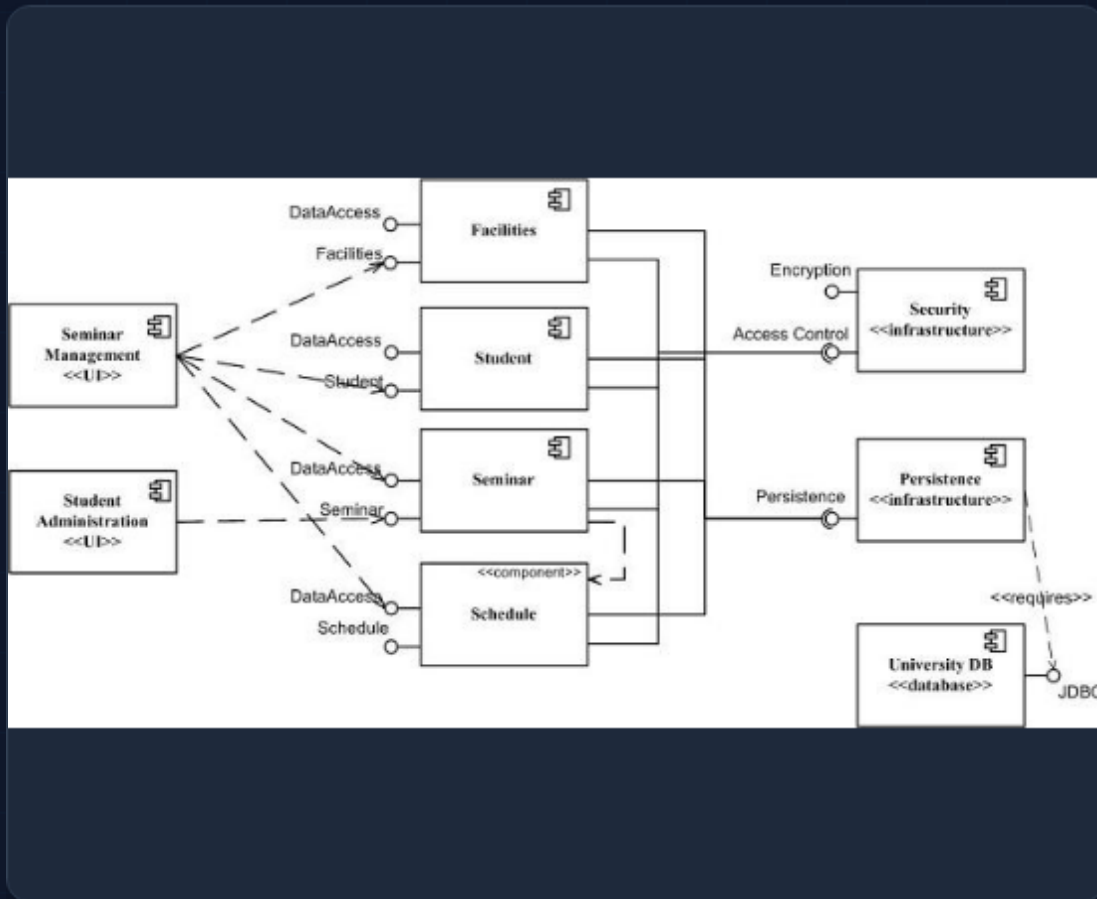
Flujos de Trabajo (Workflows)

Similares a los diagramas de flujo clásicos, pero con soporte para concurrencia (procesamiento paralelo).

- > **Nodo de Acción:** Tarea a realizar.
- > **Nodo de Decisión:** Rombo para bifurcaciones (if/else).
- > **Barra de División/Unión (Fork/Join):** Para iniciar y sincronizar hilos paralelos.
- > Útil para modelar reglas de negocio o algoritmos.



Diagramas de Componentes



Arquitectura de Software

Muestra cómo el sistema está dividido en módulos físicos de software (archivos, librerías, ejecutables) y sus dependencias.

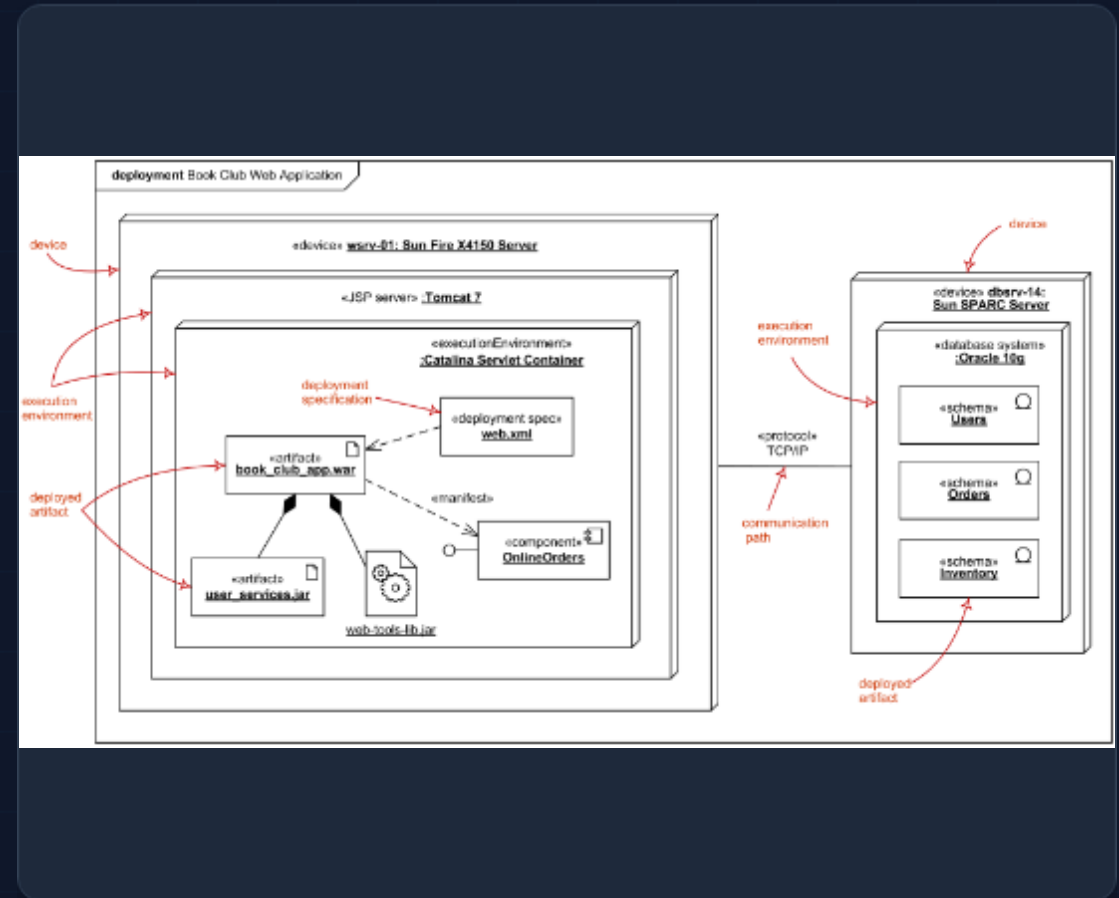
- > **Componente:** Unidad modular con interfaces bien definidas (ej: "AuthenticationService.jar").
- > **Interfaz:** Contrato que define qué servicios ofrece o requiere el componente ("Lollipop" y "Socket").
- > Ayuda a visualizar la estructura de alto nivel.

Diagramas de Despliegue

Topología Física

Modela el hardware (Nodos) y cómo los artefactos de software se distribuyen en ellos.

- > **Nodo:** Recurso de hardware (Servidor, PC, Móvil) o entorno de ejecución (Docker Container).
- > **Artefacto:** Manifestación física del software (base de datos, archivo .exe).
- > **Protocolos:** Comunicación entre nodos (HTTP, TCP/IP, JDBC).
- > Crítico para ingenieros de sistemas y DevOps.



Patrones de Diseño & UML

Soluciones Reutilizables

UML es la herramienta estándar para representar patrones de diseño (GoF). Visualizar patrones ayuda a entender soluciones complejas rápidamente.

- > **Singleton:** Clase con una sola instancia.
- > **Factory:** Interfaz para crear objetos.
- > **Observer:** Suscripción a eventos de cambio de estado.
- > **MVC:** Separación Modelo-Vista-Controlador.

Ejemplo

Visual:

Un diagrama de clases para el patrón *Observer* muestra claramente la relación "1 a muchos" entre el Sujeto y los Observadores, algo difícil de ver solo en código.

Beneficio

:

Facilita la identificación de "malos olores" (code smells) y oportunidades de refactorización.